Highlights

MLPY: A Python Framework for Spatial Machine Learning in Geosciences with Application to Heavy Metal Contamination Assessment

Francisco Parra O., Giuseppe Cavallieri

- First Python framework providing native spatial cross-validation for geoscience applications
- Standard CV underestimates prediction error by up to 31% on spatially autocorrelated geospatial data
- Case study demonstrates critical importance of spatial validation in environmental contamination assessment
- Open-source tool integrating with Python geospatial ecosystem (GeoPandas, scikit-learn)
- Practical diagnostic tools (Moran's I, variograms) for detecting spatial autocorrelation

MLPY: A Python Framework for Spatial Machine Learning in Geosciences with Application to Heavy Metal Contamination Assessment

Francisco Parra O.[1,], Giuseppe Cavallieri[1]

[a]*Department of Computer Science, Universidad de Santiago de Chile, Santiago, Chile*

Abstract

Machine learning (ML) is increasingly used in geosciences for tasks such as soil property prediction, environmental contamination assessment, and natural resource exploration. However, standard cross-validation approaches systematically underestimate prediction errors when applied to spatial data due to spatial autocorrelation—the tendency of nearby observations to be more similar than distant ones. This phenomenon, rooted in Tobler's first law of geography, invalidates the independence assumption underlying conventional validation techniques.

Currently, Python lacks an integrated framework for spatial cross-validation despite being the dominant language for ML in geosciences. While R provides solutions (e.g., `mlr3spatiotempcv`), Python geocientists must manually implement spatial resampling or accept biased performance estimates.

We present MLPY, an open-source Python framework that integrates spatial cross-validation with machine learning for geoscience applications. Through a case study of heavy metal contamination in the Meuse River

*Corresponding author

floodplain (Netherlands), we demonstrate that standard cross-validation underestimates prediction error by 31% (RMSE: 220 ppm vs. 319 ppm with spatial CV). Sensitivity analysis across different k-fold configurations confirms this finding is robust, with underestimation ranging from 15% to 33%. We provide practical guidelines for detecting spatial autocorrelation using Moran's I statistic and variogram analysis, enabling researchers to choose appropriate validation strategies.

MLPY is freely available on PyPI (`pip install mlpy-geo`) and GitHub, with comprehensive documentation and replication materials. The framework addresses a critical gap in the Python geospatial ecosystem and provides geocientists with validated tools for reliable ML model assessment in spatial contexts.

Keywords: spatial autocorrelation, machine learning, cross-validation, geospatial analysis, environmental contamination, geostatistics, Python, heavy metals

1. Introduction

Machine learning has become indispensable in modern geosciences, with applications spanning soil science [**?**], environmental monitoring [**?**], mineral exploration [**?**], precision agriculture [**?**], and natural hazard assessment [**?**]. These applications share a common characteristic: the data are inherently spatial, with measurements collected at geographic locations that exhibit spatial structure. This spatial nature introduces a fundamental chal-

lenge for standard machine learning validation approaches.

The problem stems from Tobler's first law of geography: "everything is related to everything else, but near things are more related than distant ones" [?]. In geos

cientific contexts, this manifests as *spatial autocorrelation*—the statistical dependence between observations based on their geographic proximity. Soil properties, contamination levels, mineral deposits, and groundwater characteristics all exhibit strong spatial autocorrelation across natural landscapes [? ?].

Standard cross-validation techniques assume statistical independence between training and test sets. When applied to spatially autocorrelated data, this assumption is violated: nearby observations in training and test sets provide nearly redundant information, leading to artificially optimistic performance estimates [? ?]. This overconfidence can result in poor model generalization to new locations, failed environmental assessments, and misallocation of resources in precision agriculture or mineral exploration [?].

Despite this well-documented problem, Python—the dominant language for ML in geosciences—lacks an integrated framework for spatial cross-validation. While the R ecosystem provides solutions through packages like `mlr3spatiotempcv` [?] and `blockCV` [?], Python geocientists face three suboptimal options: (1) manually implement spatial resampling strategies, (2) switch to R for validation while using Python for modeling, or (3) accept biased performance estimates from standard cross-validation.

We present MLPY, an open-source Python framework that addresses this gap by providing native support for spatial cross-validation integrated with the broader Python ML ecosystem. MLPY combines the modularity of R's `mlr3` framework [?] with Python's extensive geospatial tools (GeoPandas, PySAL, scikit-learn), enabling geocientists to:

- Perform spatial cross-validation using multiple strategies (spatial k-fold, spatial blocks, spatial buffering)
- Detect spatial autocorrelation through diagnostic tools (Moran's I, variograms)
- Systematically compare standard vs. spatial validation to quantify bias
- Integrate spatial validation into existing scikit-learn workflows
- Access practical guidelines on when spatial validation is necessary

We demonstrate MLPY through a case study predicting heavy metal contamination in the Meuse River floodplain (Netherlands), a benchmark dataset in spatial statistics [?]. Our analysis reveals that standard cross-validation underestimates prediction error by 31% (RMSE: 220 ppm vs. 319 ppm with spatial CV), with sensitivity analyses confirming this finding is robust across validation configurations. We provide practical diagnostic tools that enable researchers to detect when spatial validation is necessary for their specific datasets.

The remainder of this paper is organized as follows. Section ?? reviews spatial autocorrelation and its implications for machine learning. Section ?? surveys existing tools and positions MLPY within the geospatial Python ecosystem. Section ?? describes the MLPY framework architecture. Section ?? presents the Meuse contamination case study with comprehensive spatial validation analysis. Section ?? provides practical guidelines for applying spatial cross-validation. Section ?? discusses limitations and future directions, and Section ?? concludes.

2. Background: Spatial Autocorrelation in Geosciences

2.1. Tobler's Law and Spatial Structure

Spatial autocorrelation is the correlation among values of a variable attributable to geographic proximity [?]. In geosciences, this structure arises from physical processes: soil contamination spreads through diffusion and groundwater transport, mineral deposits form through geological processes affecting contiguous regions, and agricultural yields are influenced by spatially varying soil properties and microclimates.

Moran's I is the most widely used measure of global spatial autocorrelation [?]:

$$I = \frac{n}{\sum_i \sum_j w_{ij}} \frac{\sum_i \sum_j w_{ij}(y_i - \bar{y})(y_j - \bar{y})}{\sum_i (y_i - \bar{y})^2} \tag{1}$$

where n is the number of observations, y_i is the value at location i, $\bar{y}$ is the mean, and w_{ij} are spatial weights (typically inverse distance). Values range

from -1 (perfect dispersion) to $+1$ (perfect clustering), with $I \approx 0$ indicating no spatial autocorrelation. In environmental datasets, Moran's I typically ranges from 0.2 to 0.8, indicating moderate to strong spatial structure [**?**].

2.2. Variograms and Spatial Dependence

Variograms quantify how spatial autocorrelation changes with distance [**?**]. The empirical semivariogram is:

$$\gamma(h) = \frac{1}{2|N(h)|} \sum_{N(h)} (y_i - y_j)^2 \tag{2}$$

where $N(h)$ is the set of observation pairs separated by distance h. A typical variogram exhibits three characteristics: (1) the *nugget* (variance at zero distance), (2) the *sill* (maximum variance), and (3) the *range* (distance at which spatial autocorrelation becomes negligible). The range parameter is particularly important for designing spatial cross-validation: test locations must be farther than the range from training locations to ensure independence.

2.3. Implications for Machine Learning Validation

Standard k-fold cross-validation randomly partitions data into training and test sets. When applied to spatial data with autocorrelation range r, test observations within distance r of training observations are not truly independent. This "spatial leakage" leads to three critical problems [**?**]:

1. **Optimistic bias**: Performance metrics (e.g., RMSE, R^2) overestimate model accuracy because test observations are predictable from nearby training data through spatial autocorrelation rather than genuine model generalization.
2. **Poor transferability**: Models validated with standard CV may fail when deployed to new regions where spatial leakage is not present.
3. **Incorrect model selection**: Comparing models via standard CV may favor overfitted models that exploit spatial structure rather than meaningful predictor relationships.

Studies across multiple geoscience domains report that standard CV underestimates prediction errors by 10–50% depending on the strength of spatial autocorrelation [? ? ?]. Our Meuse case study (Section ??) provides a concrete example with 31% underestimation.

3. Related Work

3.1. Spatial Cross-Validation in R

The R ecosystem provides several packages for spatial cross-validation. The `blockCV` package [?] implements spatial block partitioning and buffered cross-validation. The `mlr3spatiotempcv` package [?] extends the `mlr3` framework with spatial and spatio-temporal resampling strategies. The `sperrorest` package [?] focuses on spatial error estimation for regression models. These tools are well-established but require R expertise, creating a barrier for Python-centric geoscience workflows.

3.2. Python Geospatial Ecosystem

Python's geospatial ecosystem includes powerful libraries: GeoPandas for spatial data structures [**?**], PySAL for spatial analysis [**?**], scikit-gstat for geostatistics [**?**], and scikit-learn for machine learning [**?**]. However, these tools operate independently with no integrated spatial cross-validation functionality. Users must manually implement spatial resampling or use workarounds like passing custom fold indices to scikit-learn's `cross_val_score`, which is error-prone and lacks standardization.

3.3. Gap Analysis

Table **??** summarizes the landscape. While R provides mature spatial CV tools and Python excels at ML and geospatial data handling, no Python framework integrates these capabilities. MLPY fills this gap by combining spatial validation strategies from R's `mlr3spatiotempcv` with Python's ML ecosystem.

Table 1: Spatial cross-validation support across frameworks

Framework	Language	Spatial CV	Python ML Integration
mlr3spatiotempcv	R	Yes	No
blockCV	R	Yes	No
scikit-learn	Python	No	Yes
GeoPandas	Python	No	Partial
PySAL	Python	Partial	No
MLPY	**Python**	**Yes**	**Yes**

4. The MLPY Framework

MLPY provides a modular architecture inspired by R's `mlr3` [?] but designed for Python's ecosystem. The framework separates concerns into composable components: tasks encapsulate data and problem type, learners wrap ML algorithms, resamplings define validation strategies, and measures compute performance metrics.

4.1. Core Components for Spatial ML

Spatial Tasks (`TaskRegrSpatial`, `TaskClassifSpatial`) extend standard ML tasks by preserving coordinate information. Coordinates are stored separately from features to prevent models from directly using location as a predictor while enabling spatial resampling strategies to access geographic information.

Spatial Resampling Strategies implement geographic partitioning:

- `SpatialKFold`: Partitions data into spatially distinct clusters using k-means clustering on coordinates, ensuring training and test sets are spatially separated.
- `SpatialBlockCV`: Creates fixed spatial blocks (grids) and uses block assignment for fold creation.
- `SpatialBufferCV`: Excludes observations within a specified buffer distance around test sets to eliminate spatial leakage.

Diagnostic Tools:

- `calculate_morans_i()`: Computes Moran's I statistic for target variables
- `create_variogram()`: Generates empirical variograms to visualize spatial dependence structure
- `diagnose_spatial_structure()`: Combines multiple diagnostics to recommend validation strategy

4.2. Example Usage

The following code demonstrates spatial cross-validation with MLPY:

```
from mlpy.tasks import TaskRegrSpatial
from mlpy.learners.sklearn import LearnerRandomForestRegressor
from mlpy.resamplings import SpatialKFold
from mlpy.measures import MeasureRegrRMSE
from mlpy import resample


# Load spatial data with coordinates
task = TaskRegrSpatial(
    data=df,
    target='zinc',
    coordinate_names=['x', 'y']
)
```

```
# Configure learner and spatial resampling
learner = LearnerRandomForestRegressor(
    n_estimators=100,
    random_state=42
)
spatial_cv = SpatialKFold(n_folds=5)

# Perform spatial cross-validation
result = resample(task, learner, spatial_cv, [MeasureRegrRMSE()])
print(f"Spatial CV RMSE: {result.score('regr.rmse'):.1f} ppm")
```

The same code with `ResamplingCV(folds=5)` instead of `SpatialKFold` performs standard cross-validation for comparison.

5. Case Study: Heavy Metal Contamination in the Meuse River Floodplain

5.1. Study Area and Dataset

The Meuse River dataset [**?**] is a benchmark in spatial statistics, containing measurements of heavy metal contamination (zinc, cadmium, copper, lead) in the floodplain near the village of Stein, Netherlands. The dataset comprises 155 observations collected in an approximately 1.2×4.0 km area with coordinates in the Dutch national grid (RD system).

Contamination originates from historical mining and smelting activities upstream, with pollutants transported by the river and deposited during

floods. Distance to the river (`dist`) and elevation (`elev`) are key predictors, as flooding frequency decreases with both distance and elevation. The dataset is publicly available and widely used for validating spatial statistical methods [?].

Our objective is to predict zinc concentration (ppm) using distance to river and elevation as predictors, evaluating whether standard or spatial cross-validation provides more reliable performance estimates.

5.2. Spatial Autocorrelation Analysis

Moran's I Statistics: We computed Moran's I for all heavy metals using inverse distance weights (Table **??**). All metals exhibit moderate positive spatial autocorrelation ($I = 0.26$–0.30), confirming the presence of spatial structure. Zinc shows $I = 0.276$, indicating that nearby observations have 27.6% more similarity than expected under spatial randomness.

Table 2: Moran's I statistics for heavy metal contamination

Metal	Moran's I	Interpretation
Zinc	0.276	Moderate positive
Cadmium	0.302	Moderate positive
Lead	0.263	Moderate positive
Copper	0.262	Moderate positive

Variogram Analysis: Figure **??** shows the empirical variogram for zinc concentration. The variogram exhibits a clear spatial structure with a range of approximately 800–1000 meters, meaning observations farther than 1 km apart can be considered spatially independent. The nugget effect is relatively

small, indicating that most variance is spatially structured rather than random measurement error. This spatial structure validates the need for spatial cross-validation.

5.3. Model Development

We used Random Forest regression [**?**] as implemented in scikit-learn, with 100 trees and default parameters. Random Forest was chosen for its robust performance on environmental datasets and minimal hyperparameter tuning requirements [**?**]. Features were standardized (zero mean, unit variance) before modeling.

5.4. Cross-Validation Strategies Compared

We compared three validation approaches:

1. **sklearn standard CV**: 5-fold cross-validation using scikit-learn's `KFold` with random shuffling (baseline)
2. **MLPY standard CV**: 5-fold cross-validation using MLPY's `ResamplingCV` (verifies MLPY matches sklearn)
3. **MLPY Spatial K-Fold**: 5-fold spatial cross-validation using MLPY's `SpatialKFold` (spatially aware validation)

Spatial K-Fold clusters the 155 observations into 5 spatially distinct groups using k-means clustering on coordinates. Each fold uses one cluster as test set and the remaining four as training set, ensuring test locations are geographically separated from training locations (Figure **??**).

5.5. Results: Performance Comparison

Table **??** presents RMSE values from each validation strategy. Standard cross-validation (both sklearn and MLPY implementations) yields RMSE ≈ 220 ppm, while spatial cross-validation produces RMSE = 319 ppm. This represents a **31% underestimation** of prediction error by standard CV compared to spatial CV.

Table 3: Cross-validation performance comparison for zinc prediction

Method	RMSE (ppm)	Underestimation
sklearn standard CV	221.7	30.5%
MLPY standard CV	220.1	31.1%
MLPY Spatial K-Fold	319.3	—

The close agreement between sklearn and MLPY standard CV (221.7 vs. 220.1 ppm) validates that MLPY's implementation is consistent with established tools. The substantial jump to 319 ppm with spatial CV reveals the true generalization error when predicting at new spatial locations.

Interpretation: Standard CV underestimates error because test observations fall within the spatial autocorrelation range (∼800–1000m) of training observations. The model leverages spatial proximity rather than learning genuine relationships between predictors (distance, elevation) and contamination. Spatial CV eliminates this leakage by ensuring test locations are farther than the autocorrelation range from training data, providing realistic performance estimates for spatial prediction.

5.6. Sensitivity Analysis

To verify this finding is not an artifact of the 5-fold configuration, we repeated the analysis with $k \in \{3, 5, 7, 10\}$ folds (Table **??**). Standard CV consistently underestimates error across all configurations, with underestimation ranging from 15.6% to 32.7% (mean: 24.7%). The 5-fold configuration is representative of the overall pattern.

Table 4: Sensitivity analysis: RMSE across different k-fold configurations

k folds	Standard CV (ppm)	Spatial CV (ppm)	Underestimation
3	235.4	279.1	15.6%
5	220.1	319.3	31.1%
7	227.3	337.9	32.7%
10	223.8	295.6	24.3%
Mean	226.7	308.0	25.9%

5.7. Practical Implications for Environmental Assessment

This 31% underestimation has significant practical consequences for environmental decision-making:

- **Risk assessment**: Contamination hotspots may be underestimated, leading to insufficient remediation efforts.
- **Sampling strategies**: Budgets for future sampling campaigns may be allocated based on overly optimistic model reliability.
- **Regulatory compliance**: Predictions used for regulatory reporting may fail to meet required accuracy when deployed to new locations.

- **Resource allocation**: Precision agriculture and mineral exploration decisions based on biased model validation may lead to suboptimal resource deployment.

These findings underscore the importance of spatial cross-validation for any geoscience application involving spatial prediction, from soil contamination assessment to precision agriculture and natural resource exploration.

6. Practical Guidelines: When to Use Spatial Cross-Validation

6.1. Decision Framework

We propose a three-step diagnostic workflow to determine when spatial CV is necessary (Figure **??**):

1. **Compute Moran's I**: If $|I| < 0.1$, spatial structure is negligible; use standard CV. If $0.1 \leq |I| < 0.3$, spatial structure is moderate; investigate further. If $|I| \geq 0.3$, strong spatial structure; use spatial CV.
2. **Analyze variogram**: If the variogram shows no clear spatial range (pure nugget effect), use standard CV. If a range exists, estimate the autocorrelation distance.
3. **Compare CV strategies**: Run both standard and spatial CV. If RMSEs differ by $> 10\%$, spatial structure significantly affects validation; use spatial CV. If difference $< 10\%$, either approach is acceptable.

6.2. Implementation with MLPY

MLPY provides diagnostic functions to streamline this workflow:

```
from mlpy.diagnostics import diagnose_spatial_structure

# Comprehensive spatial diagnostics
diagnosis = diagnose_spatial_structure(
    data=df,
    target='zinc',
    coords=['x', 'y']
)

# Returns:
# - Moran's I with p-value
# - Variogram plot
# - Recommendation: 'standard_cv' or 'spatial_cv'
```

6.3. Recommendations by Application Domain

Always use spatial CV:

- Soil property mapping (texture, nutrients, contamination)
- Groundwater level prediction
- Mineral prospectivity modeling

- Precision agriculture yield prediction

Investigate with diagnostics:

- Species distribution modeling (may have environmental rather than spatial structure)
- Climate interpolation (may have systematic rather than local patterns)
- Urban property valuation (spatial structure may be weak after accounting for covariates)

Standard CV may suffice:

- Global-scale analyses where local autocorrelation is negligible relative to domain size
- Datasets with sparse, irregular sampling where spatial structure cannot be reliably estimated

7. Discussion

7.1. Limitations

Our study has several limitations. First, we demonstrate MLPY with a single case study (Meuse dataset). While this is a benchmark dataset, additional case studies across diverse geoscience domains would strengthen generalizability. Second, we compare only Random Forest models; other algorithms (e.g., Gaussian Processes, gradient boosting) may show different

sensitivity to spatial leakage. Third, our spatial resampling strategies assume stationarity (constant spatial structure across the study area), which may not hold in heterogeneous landscapes.

7.2. Comparison with Geostatistical Approaches

Traditional geostatistics uses Kriging for spatial prediction [**?**], which explicitly models spatial autocorrelation through variogram-based covariance structures. Machine learning approaches like Random Forest do not explicitly model spatial dependence but can capture complex nonlinear relationships between predictors and targets. Recent hybrid approaches combine ML with geostatistical residual modeling [**?**], potentially offering the best of both worlds. MLPY is complementary to these approaches: spatial CV provides unbiased validation regardless of whether the underlying model is geostatistical, ML-based, or hybrid.

7.3. Future Directions

Several extensions would enhance MLPY's capabilities for geosciences:

- **Spatio-temporal CV**: Extend spatial resampling to handle time series, crucial for climate modeling and environmental monitoring.
- **Automated range estimation**: Automatically estimate variogram range and use it to configure spatial buffer distances.
- **3D spatial CV**: Support three-dimensional coordinates for subsurface modeling (e.g., hydrogeology, mineral exploration).

- **Integration with deep learning**: Adapt spatial CV for convolutional neural networks on raster/image data.

- **Additional case studies**: Validate across diverse domains (agriculture, hydrology, mineral exploration) to build empirical guidance.

8. Conclusion

Spatial autocorrelation is a fundamental characteristic of geoscience data that violates the independence assumption underlying standard cross-validation. We presented MLPY, the first Python framework providing integrated spatial cross-validation for machine learning in geosciences. Through a case study of heavy metal contamination prediction, we demonstrated that standard CV underestimates prediction error by 31%, confirming findings from the broader literature that spatial structure biases validation results.

MLPY addresses a critical gap in the Python geospatial ecosystem, providing geocientists with tools to obtain reliable model performance estimates without switching to R or manually implementing spatial resampling. The framework integrates seamlessly with existing Python tools (GeoPandas, scikit-learn) while providing practical diagnostic capabilities (Moran's I, variograms) to guide validation strategy selection.

We encourage geocientists to adopt spatial cross-validation as standard practice for any application involving spatial prediction. MLPY makes this straightforward through a simple API and comprehensive documentation.

The framework, replication materials, and example datasets are freely available, facilitating adoption and extension by the geoscience community.

As machine learning continues to permeate geosciences, rigorous validation methods are essential to ensure models generalize reliably to new locations. Spatial cross-validation is not merely a technical refinement—it is a prerequisite for trustworthy spatial prediction in environmental assessment, natural resource management, and geoscientific research.

Computer Code Availability

Name of code: MLPY (Machine Learning framework for Python with spatial support)

Developers and contact: Francisco Parra O. (francisco.parra.o@usach.cl) and Giuseppe Cavallieri (giuseppe.cavallieri@usach.cl), Department of Computer Science, Universidad de Santiago de Chile, Santiago, Chile.

Year first available: 2024

Hardware required: Any standard computer with CPU (no special hardware requirements). Tested on Linux, macOS, and Windows.

Software required: Python 3.8 or higher. Dependencies: NumPy ($\geq$ 1.20), pandas ($\geq$ 1.3), scikit-learn ($\geq$ 1.0), SciPy ($\geq$ 1.7). Optional dependencies for visualization: matplotlib ($\geq$ 3.3), seaborn ($\geq$ 0.12).

Program language: Python

Program size: Approximately 15,000 lines of Python code across core framework, learners, and utilities.

Availability and access:

- **PyPI**: Install via `pip install mlpy-geo` (version 0.1.0+)
- **Source code**: `https://github.com/franciscoparrao/mlpy`
- **Documentation**: `https://github.com/franciscoparrao/mlpy#readme`
- **Replication materials**: All scripts, data, and code to reproduce this paper's results are available in the `replication/` directory of the GitHub repository.
- **License**: MIT License (open-source, permissive)

Example usage: Comprehensive tutorials and Jupyter notebooks demonstrating spatial cross-validation workflows are available in the repository's `examples/` directory. Quick start guide and API documentation are provided in the README.

Acknowledgements

We thank the R `mlr3` development team for inspiring MLPY's modular architecture and the maintainers of scikit-learn, GeoPandas, and PySAL for providing the foundational Python geospatial ecosystem. The Meuse dataset is provided by the `sp` package in R [**?**].